

Assignment-1: Building MLP and CNN with Tensorflow and Keras

Neeil Nandal, Pranay Ram Mutini, Sohan Medendravu
Group 67

November 2025

Task 1: Learn the basics of Keras API for TensorFlow

Abstract

The assignment primarily focuses on building neural network models using TensorFlow and Keras to gain practical knowledge and optimising model performance. Through systematic experimentation, the project demonstrates the impact of data normalisation, model depth, and hyperparameter tuning on training efficiency and overall accuracy. The main challenge is a bit more original, where we need to create a CNN that can 'tell the time' from clock pictures from classification and regression approaches. Models are trained with data augmentation and regularization techniques such as dropout.

1 Introduction

Deep learning models, particularly **Multi-Layer Perceptrons (MLPs)** and **Convolutional Neural Networks (CNNs)**, are now core methods for image classification and regression, solving real-world problems from character recognition to medical diagnostics. In the first task, we aim to familiarise the Keras API through the implementation and experimentation with Multi-Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) on standard image classification datasets, **Fashion MNIST** and **CIFAR-10**. This project leans on the basics and best practices you'll find in places like Geron's "**Hands-on Machine Learning with Scikit-learn, Keras and Tensorflow.**" MLPs are effective for tabular and simple data, which they give a simple model baseline for basic image classification. The main objective of this assignment is to get hands-on learning with MLP and CNN architectures by optimising model hyperparameters and evaluating these neural network architectures with diverse datasets.

2 Experimental setup

In task 1, we used two primary datasets for this experiment:

- **Fashion-MNIST:** This data set contains 60,000 grayscale images of 28x28 size, 10,000 test images each from one of ten categories of clothes items. All values of each pixel were divided by 255 to normalise their range in $[0, 1]$.
- **CIFAR-10:** Contains 60,000 RGB 32x32 px images categorized into 10 object classes, with 50,000 training and 10,000 test samples. For CIFAR-10, data augmentation, including random rotations and shifts, was applied during training to improve generalisation.

2.1 Models and architectures

2.1.1 Multi-layer perceptron

The MLP architecture employed to classify **Fashion MNIST** comprises a fully connected network with two hidden layers. The input image is flattened into a 784-dimensional vector (28×28) before entering the network. The architecture details are as follows:

Layer type	No of units/filters	Activation function
Input Layer	784	None
Dense (Fully Connected)	128	Relu
Dense (Fully Connected)	64	Relu
Dense (Output)	10	Softmax

Table 1: MLP neural architecture

2.1.2 CNN Architecture

The CNN was composed of convolutional layers with **32** and **64** filters, followed by **maxpooling** and **batch normalization**. The feature maps were flattened before being passed to fully connected layers to produce 10-class softmax output. connected layers produce 10-class softmax outputs. The output of the convolutional layers is flattened and connected to dense layers, culminating in a softmax classification head. This design helps CNNs better capture spatial patterns than MLPs. In the Fashion MNIST architecture, there are three convolutional layers followed by max-pooling and a series of fully connected layers.

Layer details:

- Conv2D (32 Filters, 3x3 kernel size) with Relu activation, MaxPooling (2x2)

- Conv2D (64 filters, 3x3 kernel size) with Relu activation, MaxPooling (2x2)
Flatten
- Dense layer (64 units, Relu activation)
- Output layer (10 units, Softmax function)

2.1.3 Metrics and hyperparameter tuning

We had used the **sparse categorical crossentropy** as the loss function for the **MLP** and **CNN** image classification. The training process uses Adam optimizer with a learning rate between 0.0001 and 0.0005 in addition to multi-class classification-optimised categorical cross-entropy loss. Training is terminated if validation loss has not been decreasing for fifty consecutive iterations to avoid overfitting. A validation set containing 10 per cent of the training data was used to perform hyperparameter tuning. Specifically, the parameters tested were as follows:

- **Learning rate:** tuned in the interval 0.0001 and 0.0005
- **Batch size:** 32

3 Observations and results

3.1 MultiLayer perceptron (MLP)

In the multilayer perceptron, we evaluated the model and calculated the values of the training accuracy and the validation accuracy/loss values over 20 epochs. the figures are shown in the figure below:

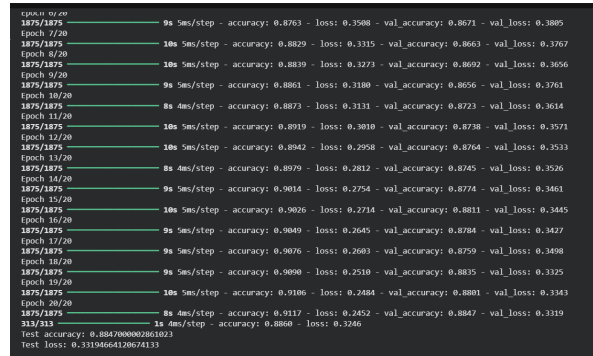


Figure 1: Training and validation accuracy/loss values of Fashion-MNIST over 20 epochs

We had used a **learning rate** of **0.0001** for the Fashion-MNIST. The training and validation accuracy have been increasing with the epochs, which shows

a strong performance of the MLPs' classification of images. We have achieved a test accuracy of **88.47** per cent for the MLP model.

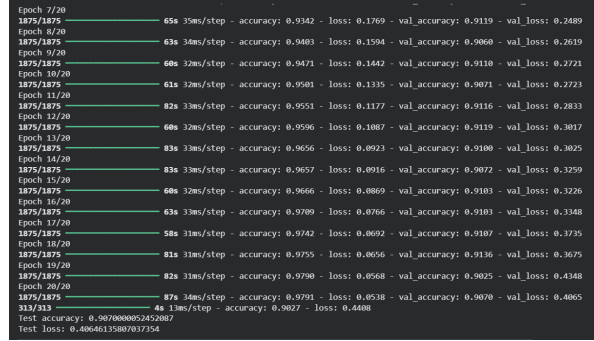


Figure 2: Training and val accuracy and loss values of the CNN model (Fashion-MNIST)

The CNN model applied to Fashion-MNIST improved on the MLP's performance, reaching a test accuracy of roughly 90.81 per cent, as shown in Figure 2. When applied to the CIFAR-10 dataset with colour images and higher perturbation, test accuracy for the deeper CNN model was around **71.84** per cent, respectively.

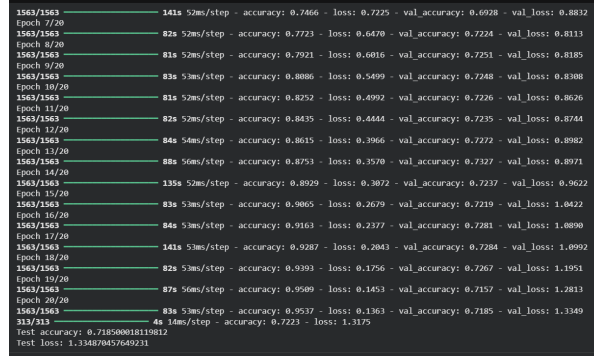


Figure 3: CIFAR-10 training and validation metrics of the CNN model over 20 epochs

For the CIFAR-10, we used a learning rate of **0.0005** with the same batch size as 32. The CNN was applied to both the Fashion MNIST and CIFAR-10 datasets. For the Fashion-MNIST dataset, the test accuracy was comparatively much higher than that of the MLP model, which achieved 90.70 per cent. The deeper CNN on CIFAR-10 achieved a test accuracy of approximately 71.84 per cent. From the data, it is evident that CNN did better than MLP in Fashion-MNIST because of its capability to extract spatial features. Accuracy on CIFAR-10 was lower than on Fashion-MNIST across all models, reflecting the

greater complexity of CIFAR-10. The learning rate of 0.0001 used for Fashion-MNIST experiments yielded stable and effective training convergence, achieving test accuracies around 87-90 per cent. This lower learning rate contributed to smooth, steady improvements without sudden divergences.

4 Discussion and conclusion

The following section discusses the results obtained to interpret the strengths and weaknesses of the models, addressing architectural choices as well as hyperparameter tuning. The analysis comparing the MLP and CNN architectures produced several technical key findings. The learning rate of 0.0001 with batch size 32 was better for the Fashion-MNIST models in this assignment, providing the best trade-off of stable training and good accuracy. For more complex datasets like CIFAR-10, small increases in learning rate while maintaining moderate batch size can help training progress effectively. The empirical results imply that CNNs can undoubtedly outperform MLPs on image classification problems. The MLP paradigm is quite basic and easy to train, but does not make use of the spatial structure present in image data. By flattening the images into a vector, spatial information is lost, and the model's complexity to learn patterns is limited. The selection of hyperparameters (learning rate, batch size, and number/size of convolutional filters) was pivotal to stability and speed of convergence during training. This low learning rate, along with a medium batch size, allowed gradual and stable training without early overfitting.

In contrast, CNNs are feedforward networks that extract hierarchical features using local convolutional filters and pooling operations to learn variation-invariant image representations, including translation invariance as well as more complex spatial relations. Task 1 established a robust foundation for subsequent modelling by showcasing the effectiveness of CNN architectures over MLPs on image classification benchmarks. It validated the use of Keras utilities, callbacks, and augmentation techniques for reliable model training. To conclude, the assignment succeeds in connecting a wider range of practical deep learning insights which will enable further research and applications involving computer vision and machine learning.

Task 2: Predicting Analog Clock Time from Images: A comparison of Label Encodings

Abstract

This study investigates predicting analog clock time from grayscale images using **convolutional neural networks (CNNs)**, comparing multiple label encodings and evaluating with a circular “**common-sense minutes**” error. Across 75×75 inputs, a multi-head architecture with scaled minute regression achieves the best validation performance at mean **13.43 minutes** and median **9.00 minutes**, substantially outperforming unscaled regression, sine-cosine circular targets, and bucketed classification. A higher-capacity 150×150 model underperforms with mean and median errors near half a cycle, indicating potential configuration or data mismatches that warrant targeted debugging. Findings emphasise the importance of output-loss scaling and decoding alignment with periodic targets for valid and precise time predictions.

5 Introduction

The goal is to infer the hour and minute from images of analog clocks by training **CNNs** under different label encodings and comparing them using a circular distance metric that aligns with human intuition about time errors. The dataset comprises **18,000** grayscale images of size 75×75 with labels [hour, minute] on a 12-hour clock, enabling systematic evaluation under consistent preprocessing and splits. A “common-sense minutes” error on a 12-hour cycle serves as the evaluation metric, reporting mean and median performance for validation and test sets. The study explores five encodings: multi-head unscaled minutes, multi-head scaled minutes, sine-cosine circular angles, and bucketed classification at 30-minute and 15-minute granularities. A further experiment scales the approach to 150×150 inputs with a deeper CNN and in-model augmentations to assess whether higher resolution improves performance.

6 Methods

6.1 Data and Preprocessing

Images are loaded from `images.npy` and labels from `labels.npy` as [hour, minute] pairs on a 12-hour clock, with image tensors normalized to $[0, 1]$, a channel dimension added if needed, and shuffling performed via a fixed RNG seed before an 80/10/10 train/validation/test split. All 75×75 experiments run on Google Colab (T4 GPU) with a standard batch size of 128 unless otherwise noted.

6.2 Evaluation Metric

Clock times are mapped to total minutes on a 12-hour cycle via $t = ((h \bmod 12) \cdot 60) + (m \bmod 60)$, and the prediction error is the minimal circular difference $d = \min(|t - \hat{t}|, 720 - |t - \hat{t}|)$. The mean and median of this common-sense minutes error are reported to summarise performance across validation or test splits.

6.3 CNN Backbone for 75×75

A shared feature extractor is used across label encodings, comprising sequential Conv2D blocks with He initialisation, Batch Normalisation, ReLU, MaxPooling, and Dropout, followed by a final Conv2D, Global Average Pooling, a Dense layer with 128 units, and Dropout. Filters progress as base, $2 \times$ base, $2 \times$ base then $4 \times$ base with base = 32 and dropout of 0.25 in intermediate blocks and 0.5 before the heads.

6.4 Label Encodings and Training

Multi-head (unscaled minutes). The head uses a 12-way hour softmax trained with sparse categorical cross-entropy and a 1D linear minute regressor trained with MSE, optimized with Adam(3×10^{-4}) and loss weights {hour : 1.0, minute : 0.2} under early stopping.

Multi-head (scaled minutes). Minute targets are scaled to $[0, 1]$ and predicted with a sigmoid-activated head to balance loss magnitudes, trained with early stopping up to 80 epochs and batch size 128, and at inference rescaled by 60 with rounding and clamping to $[0, 59]$.

Sine-cosine circular regression. Targets encode $[\cos, \sin]$ for hour and minute angles with a tanh output and MSE loss, then decode via atan2 to angles and convert to $[\text{hour}, \text{minute}]$ with rounding and clipping.

Bucketed classification (30 m and 15 m). The 12-hour cycle is discretised into $K = 720/\text{step}$ classes with softmax cross-entropy, predicting the bucket center as the decoded time.

6.5 150×150 Model

A larger model uses Conv filters $64 \rightarrow 128 \rightarrow 128 \rightarrow 256$, in-model GaussianNoise and RandomContrast during training, Global Average Pooling, Dense(256), Dropout(0.30), hour softmax and minute sigmoid heads, Adam(5×10^{-4}), equal loss weights, early stopping, and ReduceLROnPlateau.

7 Results

Table 2 summarizes validation performance on 75×75 inputs across label encodings using the common-sense minutes error.

Table 2: Validation error (minutes) by label encoding on 75×75 images; values reproduced from experimental logs.

Labeling	Mean (min)	Median (min)
Multi-Head (Unscaled)	160.99	154.00
Multi-Head (Scaled)	13.43	9.00
Sine-Cosine	176.99	175.00
Bucket 30 m	178.50	179.00
Bucket 15 m	173.26	172.00

The multi-head model with scaled minute regression is decisively superior among 75×75 approaches in both mean and median error. The 150×150 configuration yields a mean of 179.39 minutes and a median of 179.00 minutes on the held-out split, indicating failures clustered around half-cycle errors despite higher input resolution and model capacity.

8 Discussion and Conclusion

The unscaled multi-head model likely suffers from loss-scale imbalance: minute MSE in raw units dominates optimization, encouraging conservative minute predictions that translate into large circular errors even as hour classification modestly improves. Scaling minute targets to $[0, 1]$ harmonizes gradient magnitudes between hour and minute objectives, stabilising multi-task learning and enabling accurate fine-grained minute regression alongside robust hour classification. The sine-cosine approach, while theoretically appealing for periodic targets, accumulates decoding error when small angular deviations are transformed into discrete hours and minutes, and it ignores the dependency of the hour hand on minutes, degrading recovered times. Bucketed classification is bounded by coarse resolution and suffers from low accuracy over many classes, which concentrates errors near half-cycle distances under the circular metric. For 150×150 , hypotheses include label scaling or split mismatches between features and targets, altered loss balance from changing weights to $\{1.0, 1.0\}$, or adverse interactions of augmentation and regularization at higher resolution that misalign training and evaluation behaviour. Immediate checks include verifying label scaling and postprocessing paths, sanity-checking predictions on a small labelled subset, ensuring augmentation layers are inactive at validation/inference, and restoring the successful 75×75 loss weights/backbone to isolate regressions.

Among the evaluated strategies, a multi-head CNN with scaled minute regression provides the most reliable and precise time estimation on 75×75 images,

achieving a mean of 13.43 minutes and a median of 9.00 minutes on validation. Encodings that fail to align output scales and decoding with the circular evaluation metric—such as unscaled regression, naive angle embeddings, or coarse bucketization—produce large, often half-cycle errors, underscoring the need for task-aligned targets and losses. Scaling input resolution and model capacity did not automatically improve results; future work should first validate data integrity and loss balancing at 150×150 , then iterate on architecture and augmentation only after reproducing the 75×75 success.

References

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition*, O'Reilly, 2019.
- [2] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: <https://www.tensorflow.org/>
- [3] F. Agostinelli et al., “What time is it? Deep learning approaches for circadian rhythms,” *Bioinformatics*, vol. 32, no. 12, pp. i8–i17, 2016.